UQ Lab: How to Run, Read and Learn

Monte Carlo Uncertainty Lab. A checklist for running the whole project, a method for reading its code and output, and sources to learn from. Written for Anuj Bohre. Project folder: C:\Dev\uq-lab.

1. What is on your laptop and what it is for

- **JDK 25 and IntelliJ IDEA**: run and test the Java backend.
- **Node.js and npm**: run the React page in development (Vite).
- **PostgreSQL 18 and pgAdmin**: the real database, and a window onto it.
- **Docker Desktop**: runs the whole system as containers.
- **Git and GitHub**: versions of the project (tags phase-1 to phase-7).
- **Chrome**: where the page runs; its developer tools (F12) show the requests.

Everything was downloaded once. The libraries are cached in C:\Users\dell\.m2 (Maven), in frontend\node_modules (npm) and in Docker's image store. **Running needs no internet.** Only git push and opening GitHub do.

Do not run the three modes below at the same time. With 8 GB of memory they will slow the laptop, and the backend port 8080 can be used by only one of them.

2. Mode A: simplest (H2 database, nothing else to start)

1. Open IntelliJ and open the **backend** folder.
2. Open the toolbar arrow next to the run configuration, choose **Edit Configurations**, select UqLabApplication and **clear the Environment variables box**. Otherwise it will try PostgreSQL. Click Apply, then OK.
3. Click the green triangle. Wait for the log line **Started UqLabApplication**.
4. Open PowerShell and run the two lines below. Leave that window open.
5. Open Chrome at **http://localhost:5173**.

```
cd C:\Dev\uq-lab\frontend
npm run dev
```

Data is stored in the file backend\data\uqlab.mv.db. Accounts made in this mode exist only here.

3. Mode B: PostgreSQL on your laptop

1. Make sure the PostgreSQL service is running. In an **administrator** PowerShell run the first line below; the second line shows its status. After a restart it may be stopped.
2. In IntelliJ, Edit Configurations, put back the Environment variables box with two values: **SPRING_PROFILES_ACTIVE=postgres** and **UQLAB_DB_PASSWORD=** your PostgreSQL password, separated by a semicolon.
3. Start the app with the green triangle. The log should mention Flyway and the version 2 of the schema, then Started UqLabApplication.
4. In PowerShell run npm run dev as in Mode A, then open http://localhost:5173.
5. Optional: open pgAdmin to look at the tables app_user, simulation_job and flyway_schema_history.

```
Start-Service postgresql-x64-18
Get-Service postgresql-x64-18
```

The password lives only in IntelliJ's local settings and in pgAdmin. It is not in any file in the repository.

4. Mode C: everything in Docker

1 Open **Docker Desktop** and wait until it reports that it is running.

2 Stop the IntelliJ app and npm run dev (they use ports 8080 and 5173).

3 Check that the file C:\Dev\uq-lab\.env exists. It holds POSTGRES_PASSWORD and UQLAB_JWT_SECRET (32 or more characters). It is not in Git.

4 Run the commands below. Add --build only if you changed code; the first build takes several minutes.

5 Open **http://localhost:3000** (not 5173). Wait for the line **Started UqLabApplication** in the log first.

6 To stop: press Ctrl+C, then run docker compose down.

```
cd C:\Dev\uq-lab
docker compose up
# after a code change:
docker compose up --build
# when finished:
docker compose down
```

The Docker database has its own volume (pgdata). Accounts made here are separate from Modes A and B. The command docker compose down -v would delete that volume.

5. Ports and common problems

- **8080**: the backend (IntelliJ or Docker, never both). **5173**: the development page. **3000**: the Docker page. **5432**: PostgreSQL on Windows (the Docker database is not exposed on it).
- **Page says it cannot reach the server**: the backend is not running, or is still starting. Check the log for Started UqLabApplication.
- **localhost:5173 refused**: npm run dev is not running.
- **Signed out after a restart**: UQLAB_JWT_SECRET is not set in IntelliJ. This is expected and harmless locally.
- **password authentication failed**: the PostgreSQL password in the environment variable is wrong.
- **port is already allocated**: another mode is still using 8080 or 3000. Stop it.
- **Maven errors offline**: ignore Reload messages. Running still works from the cache.
- **Unsure what failed**: read the log from the first red line, not the last.

6. The taste test: how to read code and output

Do not read files from top to bottom. Follow **one request** through the system, the way the data travels.

1. **Start at the screen.** Click Run simulation with Chrome developer tools open (F12), Network tab. Click the request named jobs and read the Payload (what was sent) and the Response (what came back).
2. **Find the entrance.** In IntelliJ open JobController.java and find the method marked PostMapping. This is where the request enters the backend.
3. **Follow the calls.** Hold Ctrl and click a method name to jump into it: controller, then JobService, then the store, then the Monte Carlo engine. Write the path in your own words.
4. **Read tests as examples.** A test states an input and the exact expected output. Open JobControllerTest and read one test slowly.
5. **Use the debugger once.** Click the grey margin beside a line in JobService to set a red breakpoint. Start the app with the bug icon, run a simulation, and the program pauses. Inspect the variables. Remove the breakpoint afterwards.
6. **Read the logs.** Each line has a time, a level (INFO, WARN, ERROR), the class that wrote it, and a message.
7. **Look at the data.** In pgAdmin open a row of simulation_job and compare request_json and result_json with the page.

Five small exercises

1. Change the default number of samples in frontend\src\lib\presets.js and watch the page reload.
2. Add a log line in JobService, run a job, and find the line in the console.
3. Break one test on purpose, run it, and read the failure message. Then fix it.
4. In Swagger UI call GET /api/jobs with and without a token and compare the answers.
5. Run the same simulation twice with the same seed, then with an empty seed, and explain the difference.

How to read a number in the result

- **Mean**: the average of all sampled outputs. **Standard deviation**: how widely they spread.
- **Standard error**: the mean's precision. It falls as the number of samples grows.
- **Percentiles**: 5% means five percent of outcomes were below this value. The 5% to 95% range is a practical safe band.
- **Histogram**: how many outcomes fell in each range. The dashed line marks the mean.

7. Where to learn full stack, apart from this project

Choose one source per topic and finish it. Collecting many and finishing none is the common mistake. Check that each source is still available, since sites change.

- **HTML, CSS, JavaScript**: MDN Web Docs (developer.mozilla.org) and javascript.info.
- **React**: the official tutorial at react.dev. It is free and well written.
- **Java**: your current course, practised by typing code yourself. The Oracle Java tutorials fill gaps.
- **Spring Boot**: the guides at spring.io/guides (short and runnable) and Baeldung articles for single topics.

- **SQL and PostgreSQL**: SQLBolt (interactive), then the official PostgreSQL tutorial.
- **Git**: the free book Pro Git at git-scm.com.
- **Docker**: the Get started guide at docs.docker.com.
- **HTTP and REST**: MDN's HTTP overview. Try each endpoint in Swagger as you read.
- **Video**: freeCodeCamp's long courses on YouTube, and channels such as Java Brains, Telusko and Amigoscode. Judge a video by whether it explains why, not only what.
- **Books for later**: Head First Java for basics; Effective Java much later. Neither is urgent.

A sensible order: core Java and SQL first (they are tested in placements), then Spring Boot guides, then React basics. Use this project as the place where each new idea is tried.

8. Honest notes

- The project was built with an AI assistant. Your skill is the ability to explain and extend it. List it on your resume only after you can answer the ten questions at the end of docs\concepts-and-connections.md.
- Known weaknesses are listed in the README under Limitations: tokens in sessionStorage, no refresh tokens, no password reset, no login rate limit, polling instead of push, no deployment.
- Related files: docs\concepts-and-connections.md, docs\interview-guide.md, docs\demo-script.md, docs\phase-N-files.md.